



SOSP'26 Artifact Evaluation

for *TensorDex: A Compact, Tensor-Centric Storage System for Modern AI Models* (Paper #323)

Code	https://github.com/Antlera/TensorDex — step-by-step instructions in <code>ae/README.md</code> ; reproduced figures collect in <code>ae/RESULTS.md</code>
Data	Hugging Face dataset <code>tensor dex/tensor dex-ae-cache</code> (~8.3 GB, public), fetched by <code>make ae-cache</code> . The paper's full trace covers ~40 TB of model data and takes days to run, so we ship its pre-computed results; they are verifiable by re-deriving random samples from the raw tensor bytes we include (Tier 1)
Machine	any x86_64 Linux box: ≥ 8 GB RAM, ≥ 15 GB disk, no GPU. Please comment on HotCRP with your SSH public key and a time window if you would like us to provision the paper's eval box (AWS c6a.48xlarge) for you
Time	kick-the-tires ~5 min · full offline reproduction ~45 min (<code>make reproduce-all</code>)

Artifact at a Glance

Each claim in the paper maps to a single command that prints the measured value, the expected value, and an explicit PASS/FAIL verdict. The rest of this document walks through them: every command can be pasted into a terminal, and every output box shows the real output from our test machine.

Claim (paper)	Command	Expected
All figures derive from the published cache (Figs. 1, 2, 4, 11–16, Tab. 3)	<code>make figures</code>	38/38 charts rendered
70.5 % (FM++) / 65.1 % (TensorX) reduction is genuine (Figs. 1, 11)	<code>make ae-fmpp && make verify</code>	sampled rows re-derived from raw bytes, bit-exact
Sketch-based base selection matches exact search (Fig. 12a)	<code>make verify-recall</code>	Recall@1 = 1.000
Reduction prediction error ~1 % (Fig. 13)	<code>make verify-predict</code>	held-out MAE 1.11 %, Pearson 99.3 %
Pipeline is functional & lossless (§5)	<code>make full</code>	byte-exact round trip
FlexSplit $\approx$ ILP optimum, near-constant time (Fig. 14)	<code>make bench-fig14</code>	solver sweep re-run live
Codec throughput (Table 3, Fig. 1-right)	<code>make bench-table3-real;</code> <code>make bench-fmpp-real</code>	TensorX and FM++ parallel encode/decode, byte-exact

Throughput variants: `bench-table3` · `bench-fmpp` · `bench-baselines` (ZipNN, OpenZL, official BitX).

Installing TensorDex

TensorDex is a Python package with a Rust extension; it needs Rust ≥ 1.78 , Python ≥ 3.8 , and GNU make. From a fresh clone:

Shell

```
git clone --branch sosp26-ae https://github.com/Antlera/TensorDex.git
cd TensorDex
# Rust toolchain, if you don't already have one:
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh -s -- -y
source "$HOME/.cargo/env"
# a virtualenv (stock Ubuntu ships neither venv nor a usable system pip):
sudo apt-get install -y python3-venv # once; or use uv instead
python3 -m venv .venv && source .venv/bin/activate
# optional, saves ~5 GB: CPU-only torch (no GPU is ever used)
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install . # builds the Rust extension
pip install -r ae/requirements-ae.txt # figure + verification deps
```

The repo also ships a Dockerfile with everything (including the optional FM++ codec) preinstalled:

Shell

```
docker build -t tensordex-ae .
docker run --rm tensordex-ae # runs `make check` inside the container
docker run -it --rm -v tensordex-cache:/tensordex/ae/cache tensordex-ae bash
```

Kick the Tires (~5 minutes, no download)

`make check` confirms the extension built correctly: it cross-checks the XXH3-128 content hash against an independent library (the ids in the shipped cache are these hashes) and runs the offline unit tests.

Shell

```
make check
```

Output

```
OK build + XXH3-128 hash
..... [100%]
18 passed, 1 skipped in 2.78s
OK build verified -- no download needed for this check
```

`make full` then runs the whole pipeline on a synthetic base + fine-tune pair — ingest, tensor-level dedup, delta compression, and an integrity-verified pull — entirely offline:

Shell

```
make full
```

Output

```
---- 1. ingest two models (tensor-level dedup) -----
  ingested base      (3 params)
  ingested finetune  (3 params)
  -> 2 models, 6 logical params, but only 4 unique tensors stored
      (embed + norm are shared -> stored once)

---- 2. compress the fine-tune against the base (raw -> delta) -----
  attn.weight: 1.00 MiB -> 0.10 MiB (10.0x) via tensorx delta

---- 3. the delta-base graph lives in SQL (tensor_deltas) -----
  cd2a67cdf9de... --delta(tensorx)--> b35165b4faf5...
  protected bases (gc-safe): ['b35165b4faf5...']

---- 4. pull the fine-tune back, with integrity verification -----
  pull --verify -> 3 tensors, byte-exact round-trip: True
  (attn.weight was delta-decoded against the base, then re-hashed)

---- 5. sharded pull (bounds peak memory; HF-style index) -----
  --max-shard-size 8MiB -> 3 shards + index.json

---- 6. gc keeps the base alive because a delta still needs it -----
  removed 'base' model; gc deleted 0 tensors, protected 1 delta base(s)
  finetune still reconstructs after gc: True

demo complete [ok]
```

If both commands pass, the build is good — everything that follows is just data + compute.

Getting the Data

The evaluation cache lives in a public Hugging Face dataset (no token needed): `results.db` — the 11.4 M-pair compression cache every figure is drawn from — plus the raw tensor blobs for Tier-1 verification and the staged chart inputs.

Shell

```
make ae-cache          # everything (~8.3 GB) -> ae/cache/
make ae-cache-figures  # figures-only alternative (~6 GB, skips the blobs)
```

This populates:

Output

```
ae/cache/
  results.db                11.4M-pair compression cache (5.6 GB)
  sample_blobs/xx/yy/<id>.safetensors  raw tensors for Tier-1 verify (~2 GB)
  data/tensordb_s3/metadata.db  tensor sizes + model->tensor map
  model_hub_crawl/ tests/output/ compression_data/ ...  chart inputs
```

Reproducing Experiment Results

NOTICE: Throughput/QPS panels (Fig. 1 right, Fig. 12b/c, Table 3) replot values measured on the paper's evaluation machine (c6a.48xlarge); absolute throughput on your box will differ — re-measure it with `make bench-table3-real` (below). Every ratio-bearing result below is re-derived, not replot-

ted.

Every figure from the cache (Tier 0)

Shell

```
make figures          # ~10 min -> ae/figures/*.png    (FMT=pdf for paper-quality)
```

Output

Rendering 38 charts from ae/cache/results.db

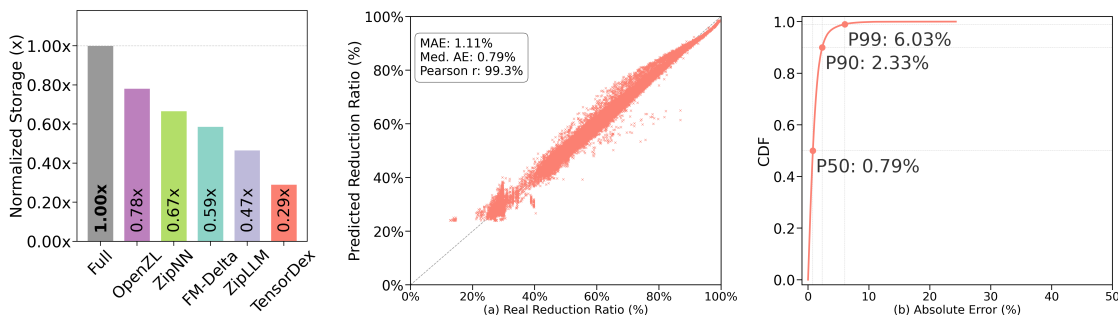
```
ok  algo_bench_q_proj.png
ok  algo_bench_v_proj.png
ok  bcs_memory.png
...
ok  zipllm_real_bar.png
ok  zipllm_real_cdf.png
```

=====

rendered 38 figures -> ae/figures

ae/RESULTS.md lays the figures out against the paper's evaluation section, and ae/FIGURE_MAP.md maps every chart id to its paper figure. `make report` builds a self-contained single-file `ae/results.html` with every figure embedded (`make serve` hosts it on `localhost:8000`).

Two spot-checks against the paper — Fig. 1 (left): TensorDex stores the corpus at $0.29\times$ (70.5% saved), below every baseline; and Fig. 13: predicted vs. real reduction ratio hugs the diagonal:



Bit-exact sample verification (Figs. 1 and 11, Tier 1)

The cache has 11.4M rows; rather than recompress all of them, you draw a random sample — *with a seed you choose* — from the ~ 342 rows the shipped raw-blob bundle covers, and every sampled row is re-derived from the raw tensor bytes shipped in the cache: the content ids are re-hashed (XXH3-128) and the TensorX / FM++ compression ratios are recomputed and compared against the cached values, bit for bit.

First build the optional FM++ codec (the codec behind the 70.5% result; vendored FM-Delta lib, x86_64-linux — skip this line and `verify` still checks TensorX):

Shell

```
make ae-fmpp          # maturin develop --release --features fmpp
make verify           # N=200 default; or:
```

```
python ae/verify_sample.py --n 300 --seed 7      # your own seed and size
make verify-fmpp-roundtrip # synthetic FM++ round trip; no cache needed
```

Output

```
Scanning blob store ae/cache/sample_blobs ...
 326 tensor blobs available
 342 cached pairs fully covered by the blob sample

Verifying 200 random pairs (seed=0, codecs: TensorX (level 1) + FM++)
[ 1/200] OK 9a91c437e6a6... id=[ok] tratio got=0.345300830 exp=0.345300830
[ 2/200] OK 09fef6d3451d... id=[ok] tratio got=0.336914062 exp=0.336914062
[ 5/200] OK 72034a8240d1... id=[ok] tratio got=0.421907902 exp=0.421907902
                                     fratio got=0.359875679 exp=0.359875679
[ 6/200] OK 1d91936611eb... id=[ok] tratio got=0.068219866 exp=0.068219866
                                     fratio got=0.034877232 exp=0.034877232
... (194 more)

=====
content-id checks : 200/200 exact
TensorX ratio      : 199/200 bit-exact (99.5%) (1 legacy rows)
FM++ ratio         : 94/94 bit-exact (100.0%)

RESULT: PASS  cache reproduced from raw bytes
=====
```

Because the cache is keyed by content hash, a matching id ties the row to real tensor bytes, and a matching ratio proves the cached number is what the codec actually produces on those bytes. You pick the seed, so we could not have pre-selected favorable rows. (The one non-bit-exact row above is a documented *legacy row* — it predates the final codec build and is reported with its timestamp; its content ids still match, and the run still PASSES. See Notes.) The separate FM++ round-trip command needs no cache: it generates deterministic synthetic base/target tensors, creates fresh deltas, decodes them, and requires byte-exact reconstruction.

TensorSketch Recall@1 (Fig. 12a)

Re-runs base selection twice over fingerprints recomputed from the shipped blobs — exact brute-force search (ground truth) vs. the HNSW index TensorDex uses — and reports how often they pick the same delta base:

Shell

```
make verify-recall
```

Output

```
Recomputing BCS fingerprints (d=2, w=1024) for 326 tensors ...
Greedy base-selection: brute-force (ground truth) vs hnswlib (HNSW) ...

=====
tensors 326 . bases 67 . attached 259
TensorSketch Recall@1 (HNSW base == brute-force base) : 1.0000

RESULT: PASS  matches the paper's ~1.0 recall (>=0.99)
=====
```

Reduction-ratio prediction (Fig. 13)

This is reproduced *as an experiment, not a replot*: TensorPred’s hybrid model is re-fit from the cached (bcs_dist, ratio) pairs by ordinary least squares — no stored coefficients — trained on a random half and evaluated on the held-out half:

Shell

```
make verify-predict
```

Output

```
pairs : 5,768,502 (bcs_dist > 0, aratio not null, bytes_in > 100KB)
```

```
recovered hybrid coefficients (train split):
```

```
c0=-6.0383 c1=+0.1553 c2=+0.6240 c3=+0.0135
```

	MAE	Med. AE	Pearson	
held-out (50% test)	1.11%	0.79%	99.3%	(n=2,884,251)
paper (Fig 13)	1.11%	0.79%	99.3%	

```
provenance vs stored pred_ratio: max|delta|=0.0001 Pearson=100.000%
```

```
PASS: held-out prediction MAE 1.11% (< 1.5%), Pearson 99.3% (> 99%)
```

The recovered model also matches the cache’s stored pred_ratio column to 10^{-4} — confirming that column *is* this model, not a hand-tuned constant.

FlexSplit vs. ILP / Primal–Dual (Fig. 14)

Actually runs the three facility-location solvers over the cached pairwise ratios, sweeping model counts, and re-renders both panels. Expect FlexSplit within a few points of the ILP optimum at near-constant time, while ILP time grows super-linearly.

Shell

```
make bench-fig14      # 15-25 min; ILP needs gurobipy + a (free academic)
                      # Gurobi license -- skipped automatically if absent
```

Codec throughput (Table 3, Fig. 1-right)

Re-measures TensorX with the paper’s exact setup: the real Qwen2.5-7B/-Instruct pair, 2 MB sub-chunks, zstd level 1, pure codec compute across all cores. Below is this command’s output on a c6a.48xlarge during packaging — reduction matches the paper exactly and throughput lands on the published numbers. Baseline rows in Table 3 cite each system’s own published figures (ZipLLM, ZipNN, OpenZL); ae/bench_baselines.py re-runs the external tools. The Docker image includes the submission-time versions, ZipNN 0.5.3 and OpenZL v0.1.0.

Shell

```
make bench-table3-real # ~30 GB model download on first run, ~32 GB RAM
make bench-table3      # synthetic variant, no download
make bench-fmpp-real   # native tensor-pair parallel FM++ + decoder
make bench-fmpp        # synthetic FM++ variant, no download
```

```
make bench-baselines      # ZipNN 0.5.3 / OpenZL v0.1.0
```

Output

```
TensorX codec throughput - real model pair (14525 MB across 7481
chunks, 192 threads, zstd L1)
  reduction    0.406x (59.4% saved)   round-trip byte-exact [ok]
  compress     avg   24.3 GB/s   peak   24.7 GB/s
  decompress   avg   28.6 GB/s   peak   29.2 GB/s

paper reference (c6a.48xlarge, 192 vCPU, real Qwen2.5-7B pair):
  22.9 / 28.4 GB/s @ 59.4%
RESULT: PASS   round-trip byte-exact; throughput measured
```

The FM++ benchmark keeps every arithmetic-coded tensor stream intact and uses Rayon to schedule independent tensor pairs across cores, matching the original aggregate-throughput methodology. Before timing, it materializes every delta, decodes it against its base, and requires byte-exact reconstruction. The real variant uses the same Qwen2.5-7B Base / Instruct pair and needs roughly 40 GB of RAM. Aggregate throughput depends on the CPU count and the tensor-size distribution; a single model pair can exhibit a long tail from its largest embedding tensors.

Everything in one command

After `make ae-cache`, this runs all of the above (except the solver benchmark) plus the HTML report, back to back:

Shell

```
make reproduce-all      # Tier 0 + Tier 1 + Tier 2 demo + report, ~30-45 min
```

Running on Your Own Models

The same pipeline works on live Hugging Face repos — pick a base and a fine-tune and TensorDex reports the achieved storage reduction:

Shell

```
python ae/run_full.py --mode hf \
  --models Qwen/Qwen2.5-0.5B Qwen/Qwen2.5-0.5B-Instruct
python ae/render.py --only reduction_global --format pdf    # re-render one chart
python ae/run_full.py --recipe    # the full 40 TB / 2,890-model trace recipe
```